

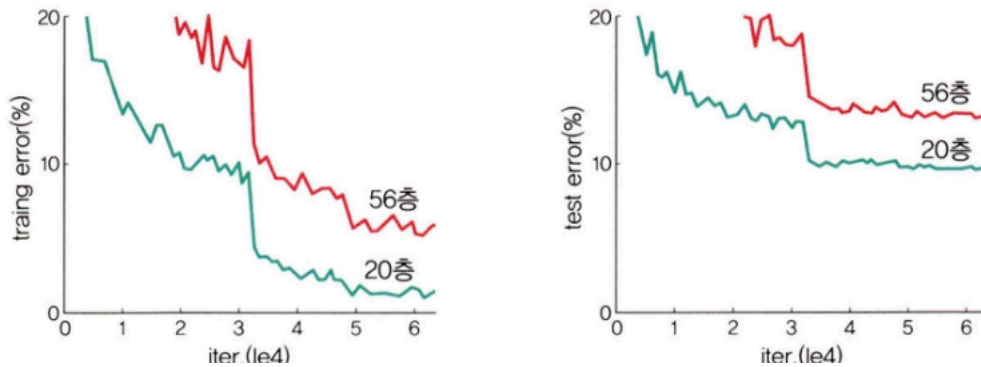
6.1.5 ResNet

마이크로소프트에서 개발한 알고리즘, 논문: Deep Residual Learning for Image Recognition

신경망을 효과적으로 학습하기 위한 방법으로 **residual**개념 고안

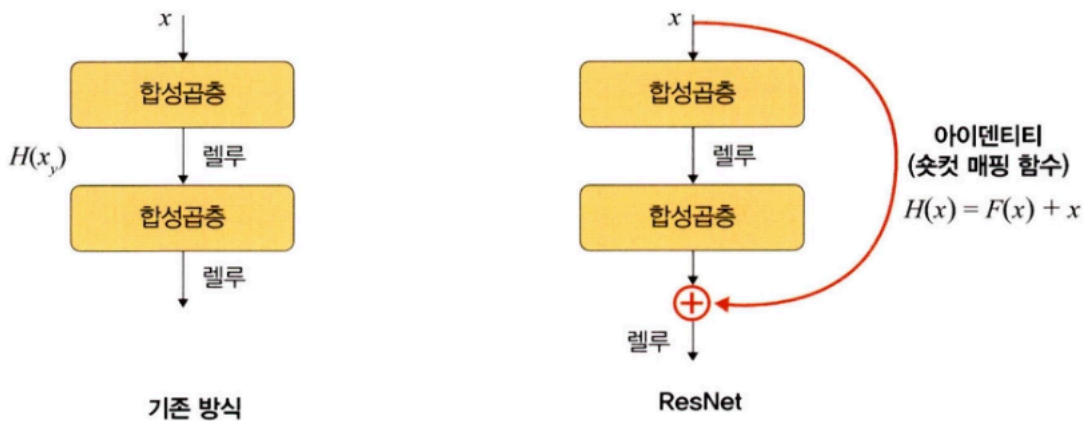
신경망은 깊이가 깊어질 수록 성능이 좋아지다가 -> 일정 단계에 다다르면 오히려 성능 나빠짐

▼ 그림 6-24 네트워크 56층이 20층보다 더 나쁜 성능을 보임



이런 문제를 해결하기 위해 **Residual Block** 도입 : 기울기가 잘 전파될 수 있도록 숏컷 만들
층이 22개인 GoogLeNet에 비해 ResNet은 152개의 층으로 기울기 소멸 문제 발생 가능

▼ 그림 6-25 ResNet 구조



Block: 계층의 묶음, 합성곱층을 하나의 블록으로 묶은 것(residual block)

▼ 그림 6-26 ResNet 모델 전체 네트워크



<- 색상별로 블록 구분

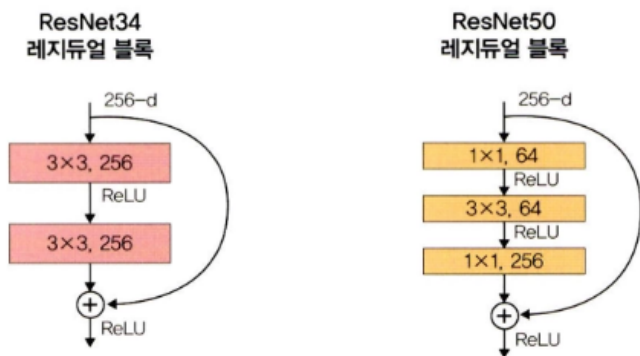
ResNet: Residual block을 여러개 쌓은 것

이렇게 계층을 쌓아 늘리면 파라미터가 계속 증가하는 문제 발생

Ex) ResNet34: 34개의 합성곱층 & 16개의 블록으로 구성: if 첫번째 파라미터: 1152K -> 전체 파라미터: 2만 1282K

=> 병목 블록 도입

병목 블록

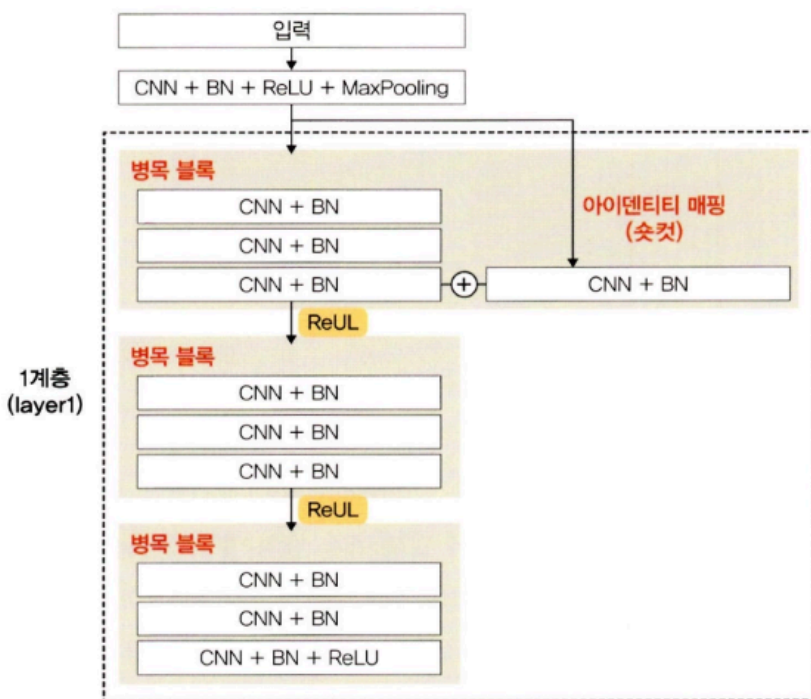


1x1 : 합성곱층의 채널 수를 조절하면서 차원을 줄였다, 늘렸다 하는것이 가능하다는 특징 -> 이를 통해 파라미터 수 감소시킴

아이덴티티 매핑(Identity mapping) / 숏컷, 스킵 연결

아이덴티티 매핑: 입력x가 어떤 함수를 통과하더라도 다시 x라는 형태로 출력되도록 함

▼ 그림 6-28 아이덴티티 매핑(숏컷)



<- 아이덴티티 매핑

```
def forward(self, x):
```

```
    i = x
```

```
    x = self.conv1(x)
```

```
    x = self.bn1(x)
```

```
    x = self.relu(x)
```

```
    x = self.conv2(x)
```

```
    x = self.bn(x)
```

입력 x는 i라는 변수에 저장해서

.

.

.

.

.

합성곱층 통과

```
if self.downsample is not None:
```

```
    i = self.downsample(i)
```

```
x+=I #아이덴티티 매핑이 적용되는 부분,
```

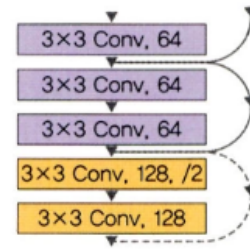
```
x = self.relu(x)
```

마지막에 I 더해줌

```
return x
```

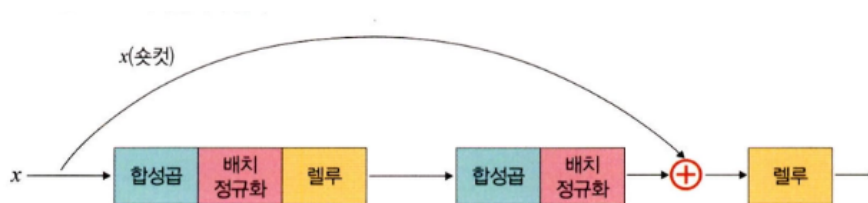
다운샘플링

특성 맵(feature map)을 줄이기 위한 것, 풀링과 같은 역할
좌측의 ResNet에서 노란 영역에서 64개였던 기존의 채널 수가 128개로
늘어났고, /2로 첫번째 블록에서 합성곱층의 스트라이드가 2로 늘어나
(28, 28, 64) 였던 맵의 형상이 (14, 14, 128)로 바뀐다는것을 알 수있음



형태가 다른 맵들이 있을때는 형태를 맞추기 않으면 아이덴티티 매핑을 할 수 없음 -> 다운샘플링 필요

입력과 출력의 형태가 같도록 맞추려면 스트라이드2를 가진 1x1합성곱 계층을 하나 연결해 주면 됨.
입력&출력이 같은 것을 **아이덴티티 블록**



입력&출력 차원이 동일하지 않아 입력 차원을 출력차원에 맞춰 변경해야 하는것을 **프로젝션 숏컷/합성곱 블록**



ResNet: VGG19구조를 뼈대로, 합성곱층들 추가 + 숏컷 추가

▼ 그림 6-32 VGG19와 ResNet 비교

